Unit 5 - Hashing

CS 201 - Data Structures II

Spring 2024

Habib University

Syeda Saleha Raza

Let’s talk about dictionaries (Maps)…

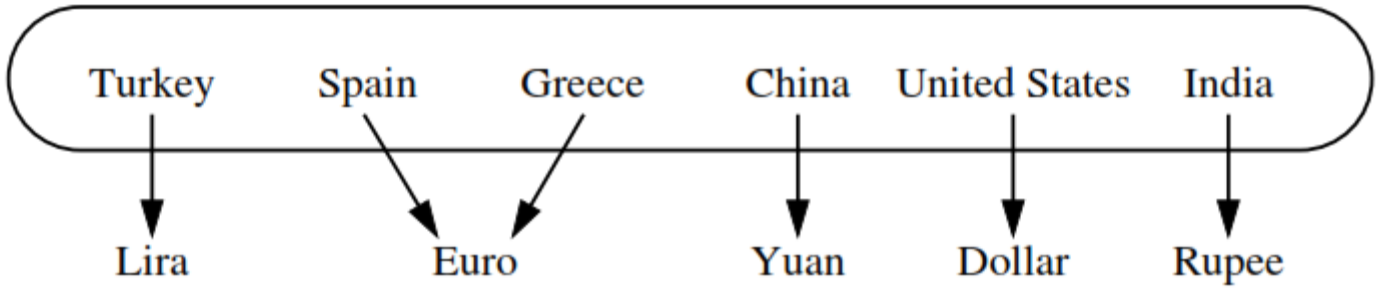
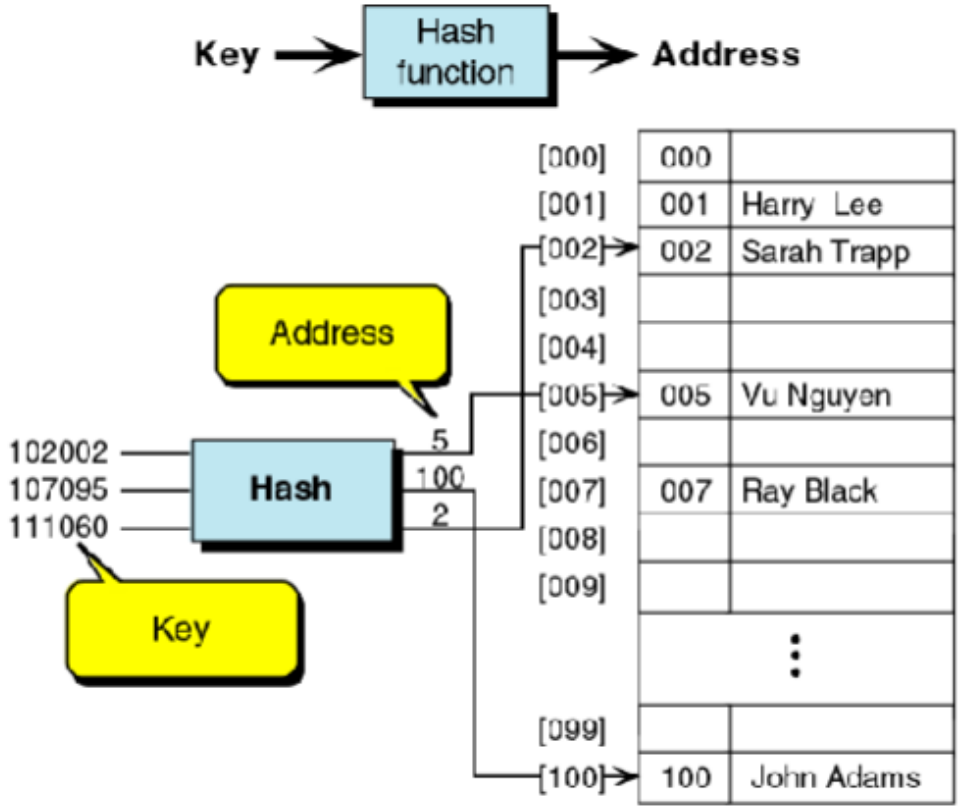


Figure 10.1: A map from countries (the keys) to their units of currency (the values).

How are they implemented?
and
Why are they fast?

Hashing



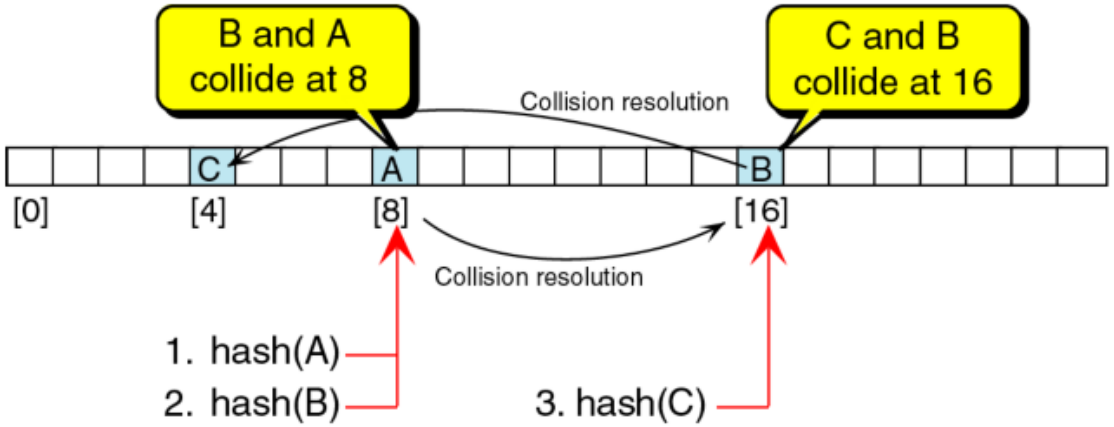
Visualiser: https://www.cs.usfca.edu/~galles/visualization/OpenHash.htm

Hashing

- Generating address from a key
- Key is a record's property on which a record is stored in memory (called **prime area)**
- A hash search is a search in which the key is modified (using an algorithm) into a memory address (called **home address)**
- The conversion process of **converting a key to a memory address is called Hashing**

Collisions

- A hash **collision occurs when an key is inserted into an occupied memory location**
- More than one keys that hash to the same location are called **synonyms.**
- Calculation of a memory address and its test for collision is called a **probe.**

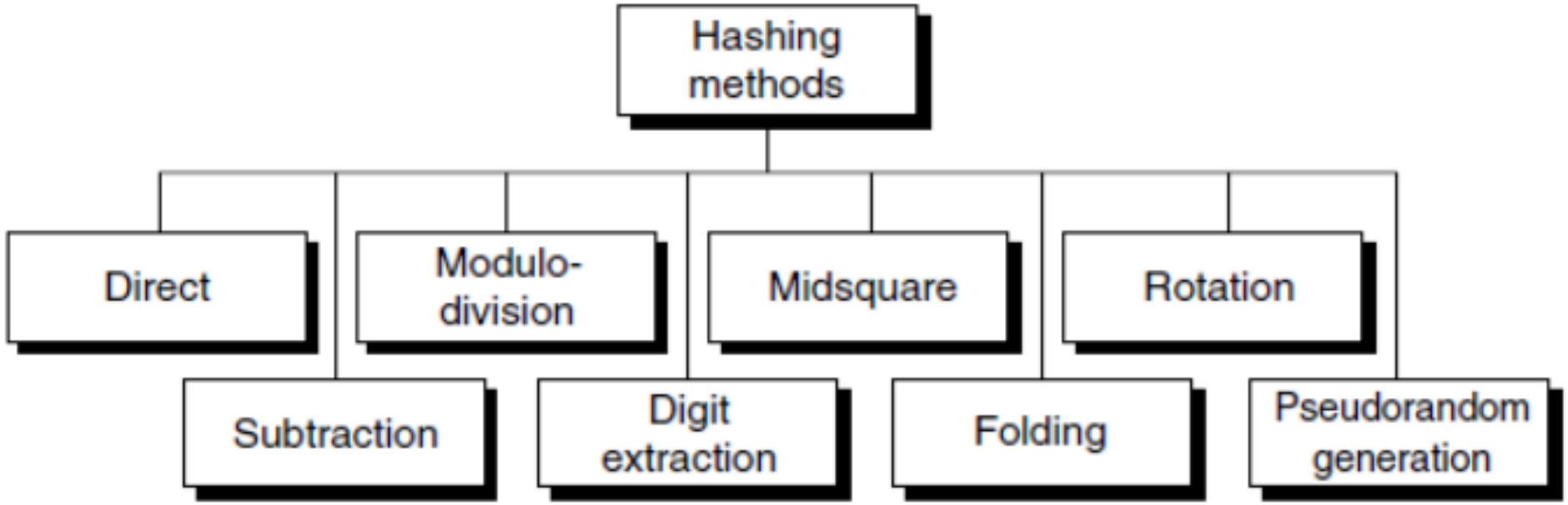


Hash Functions

Hash Function

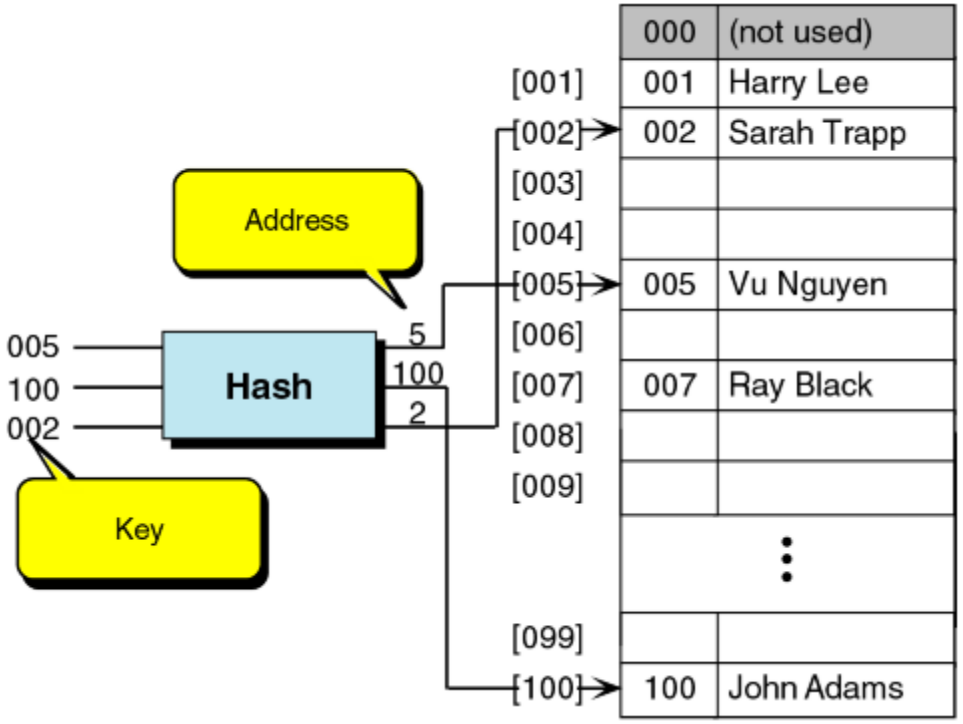
- The hash function maps keys to indices in the hash table.
- Hash function should be such that:
  - the indices are uniformly distributed (will use all of the range evenly)
  - there is low probability of collision (related partly to the previous)
  - computationally fast

Hash Function



Direct Method

- The key becomes the address.

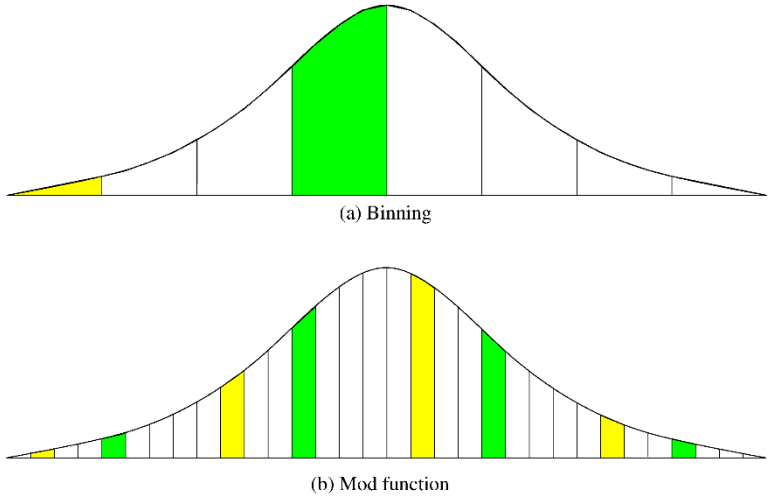


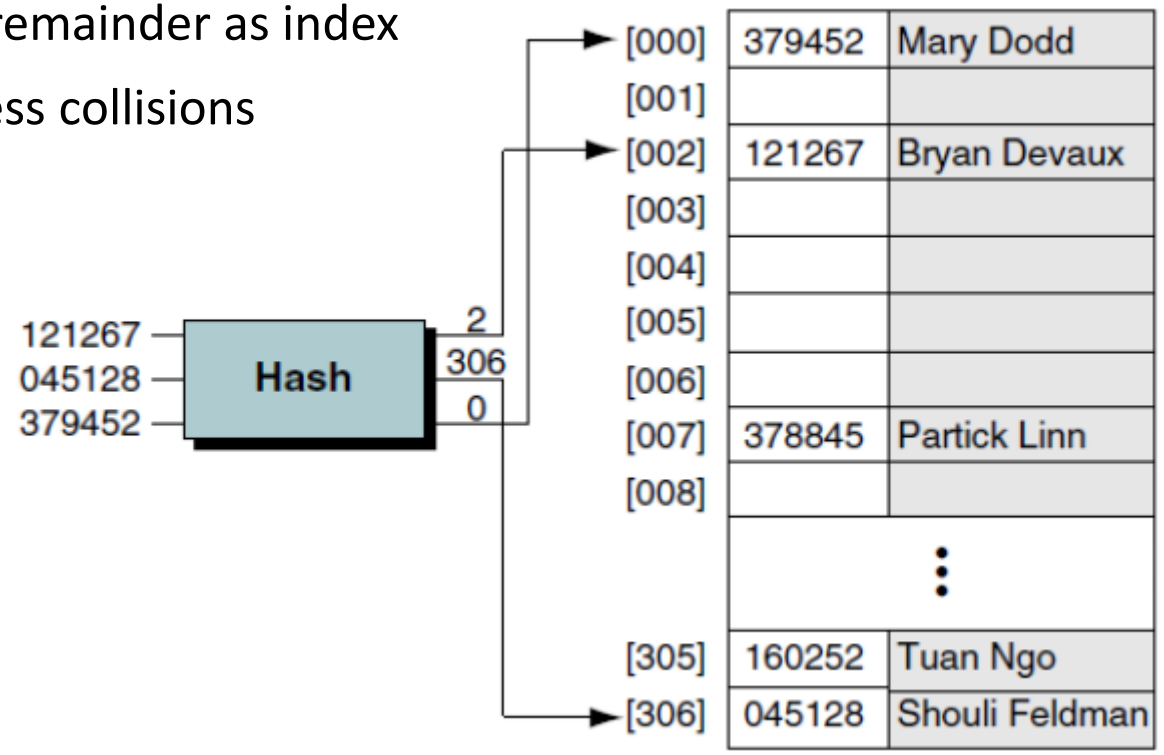
Subtraction Method

- If keys do not start from 0 or 1, you get an index by subtracting a constant value from the key
- The direct and subtraction hash functions both **guarantee a search effort of one with no collisions.**
- They are one-to-one hashing methods that is only one key hashes to each address.

Modulo Division Method

- Also known as division remainder or modulo-division method
- Divides the key by list/array size and uses the remainder as index
- Works with any size but prime numbers give less collisions





[7.5. Sample Hash Functions — An Introduction to Data Structures and Algorithm A (vt.edu)]

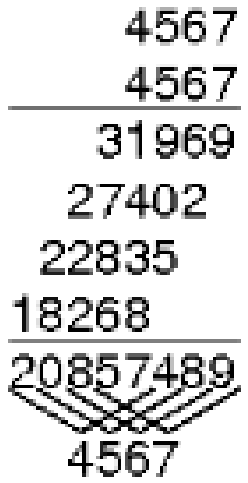
Digit Extraction Method

- Extract certain digits and then use the obtained number as address
- Assuming that we have an array size of 1000 (indices from 0 to 999) we can use three digits
- Example: (extracting 1^{st} , 3^{rd} , and 4^{th} digit)
  - **145674 => 156**
  - **344565 => 345**
  - **132554 => 125**

Mid-square Method

- The mid-square method squares the key value, and then takes out the middle r bits of the result, giving a value in the range 0 to 2r−1
- This works well because most or all bits of the key value contribute to the result.

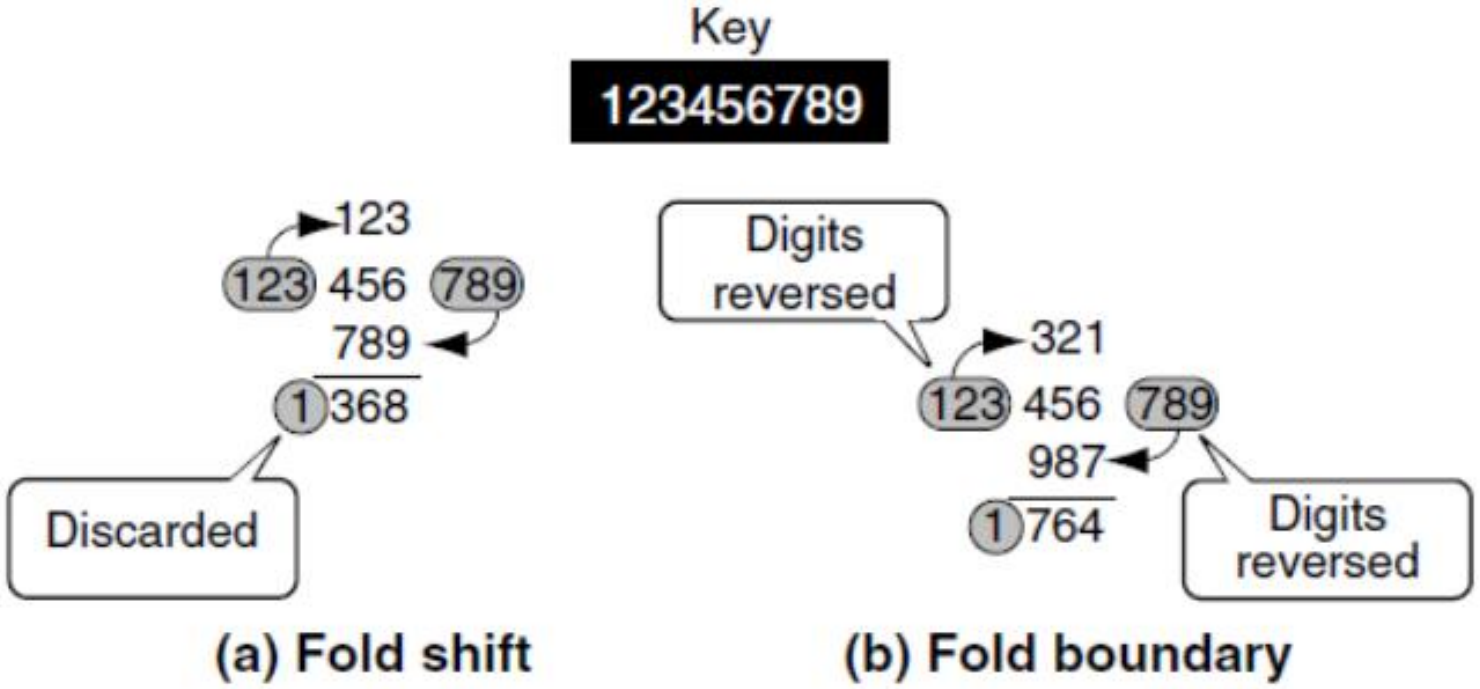
Input	Square	Index/Address
121	1**464**1	464
365	13**322**5	322
749	56**100**1	100



Folding

- Two methods
  - Fold shift
  - Fold boundary
- Fold shift
  - Key is divided into parts whose size matches the required address size
  - The left and right parts are shifted and added to the middle part
- Fold boundary
  - The key is divided into parts as in fold shift
  - The digits of left and right parts are reversed and then added to the middle part
- In both folding methods, overflowing digits are discarded

Folding


Key
123456789
123
123 456 789
789
1368
Discarded
(a) Fold shift
Digits reversed
321
123 456 789
987
1764
Digits reversed
(b) Fold boundary

Multiplication Method

- This method involves the following steps:
  1) Choose a constant value A such that $0 < A < 1$.
  2) Multiply the key value with A.
  3) Extract the fractional part of kA.
  4) Multiply the result of the above step by the size of the hash table i.e. M.
  5) The resulting hash value is obtained by taking the floor of the result obtained in step 4.

     h(K) = floor (M (kA mod 1))

     Example k = 12345
     A = 0.357840
     M = 100

     h(12345) = floor[100 (12345*0.357840 mod 1)]
     = floor[100 (4417.5348 mod 1)]
     = floor[100 (0.5348)]
     = floor[53.48]
     = 53

Collision Resolution Methods

Chained Hashtable

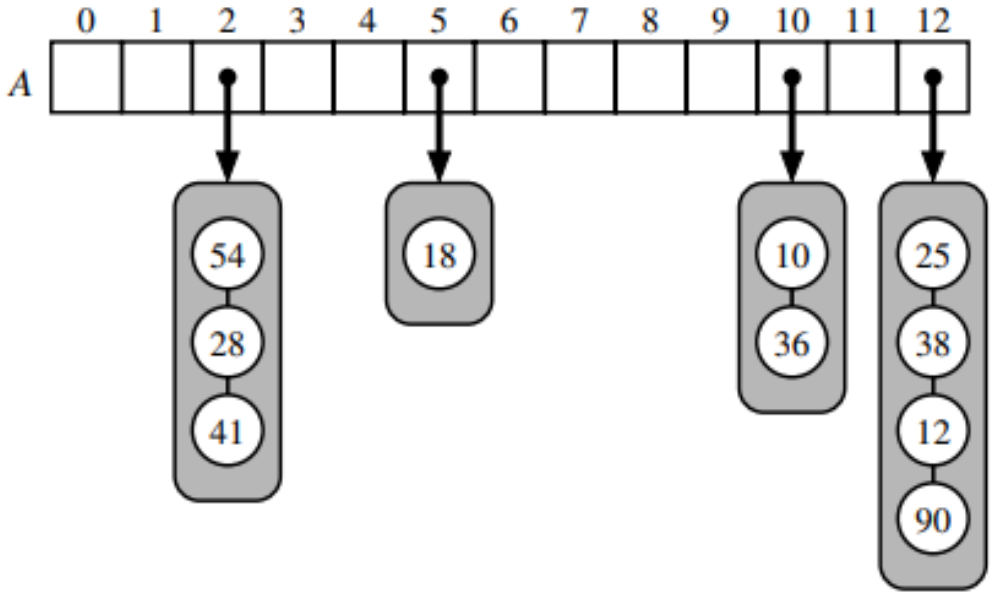


Figure 10.6: A hash table of size 13, storing 10 items with integer keys, with collisions resolved by separate chaining. The compression function is $h(k) = k \bmod 13$. For simplicity, we do not show the values associated with the keys.

Chaining - Example

- x: 5 , 9 , 16, 4, 12, 7, 8 , 3
- Hash function: (x-3) % 5

ChainedHashTable - Exercise

- X: 4, 20, 39, 17, 29, 34, 11,60, 45, 58, 48, 12,
- Hash function: (x-3) % 11

Open Addressing

Open Addressing

- The ChainedHashTable data structure uses an array of lists, where the ith list stores all elements x such that hash(x) = i.
- An alternative, called *open addressing is to store the elements directly in an array, t, with each* array location in t storing at most one value.

$$norm(h(K) + p(1)), norm(h(K) + p(2)), \ldots, norm(h(K) + p(i)), \ldots$$

Linear Probing

- The main idea behind linear probing is that we would,
  - like to store the element x with hash value i =hash(x) in the table location t[i].
  - If we cannot do this (because some element is already stored there) then we try to store it at location t[(i +1) mod t.*length];*
  - *if that is* not possible, then we try t[(i+2) mod t:*length], and so on, until we find* a place for x.

$$h(k,i) = (h'(k) + i) \bmod m$$

Collision Handling - Linear Probing

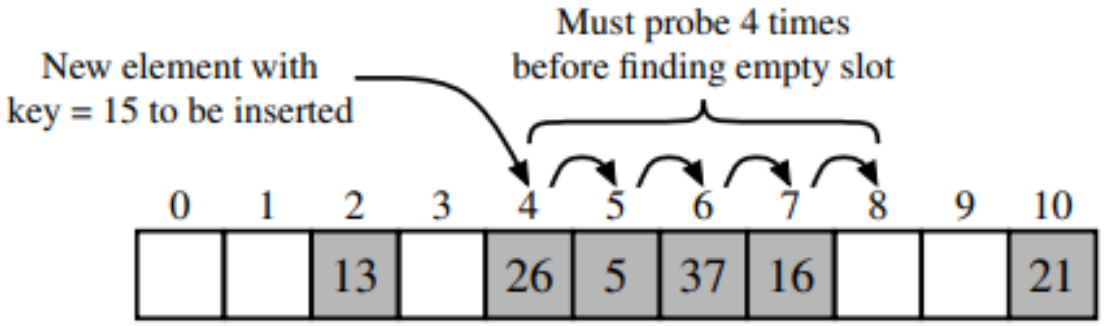


Figure 10.7: Insertion into a hash table with integer keys using linear probing. The hash function is $h(k) = k \bmod 11$. Values associated with keys are not shown.

- Probability of empty cells to be filled: 1/M
- Probability of cells following a cluster to be filled: (sizeof(cluster) + 1)/M

Collision Handling – Quadratic Probing

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \ ,$$

- Although using quadratic probing gives much better results than linear probing, the problem of cluster buildup is not avoided altogether, because for keys hashed to the same location, the same probe sequence is used. Such clusters are called **secondary clusters**. These secondary clusters, however, are less harmful than primary clusters.

Quadratic Probing

- An alternate scheme of quadratic probing is:

$$h(K), h(K)+1, h(K)-1, h(K)+4, h(K)-4, \ldots, h(K)+(TSize-1)^2/4,$$

$$h(K)-(TSize-1)^2/4$$

Double Hashing

- The problem of secondary clustering is best addressed with double hashing.
- This method utilizes two hash functions, one for accessing the primary position of a key, h, and a second function, h, for resolving conflicts. The probing sequence Double hashing uses a function of the form:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

  where both h1 and h2 are auxiliary hash functions.
- In case of double hashing, if two keys k_1 and k_2 are hashed on same index, their probing sequence can still be different as that depends upon $h_2(k)$.

Double Hashing

- There are important restrictions on h_2. Most importantly, the value returned by h_2 must never be zero (or M) because that will immediately lead to an infinite loop as the probe sequence makes no progress.
- However, a good implementation of double hashing should also ensure that all of the probe sequence constants are relatively prime to the table size M. For example, if the hash table size were 100 and the step size for linear probing (as generated by function h2) were 50, then there would be only one slot on the probe sequence. If instead the hash table size is 101 (a prime number), than any step size less than 101 will visit every slot in the table.

Resizing a hash table

Hashing Exercise

- Numbers to be inserted in a hash table of size 11:

{12, 44, 13, 88, 23,94, 11, 39, 20,16,5}

h(x) = (3x + 5) % 11

1. Using chaining
2. using linear probing
3. using quadratic probing
4. using double-hashing

   h`(x) = 7 – (x % 7)

5. Let’s resize the hashtable obtained in part (2)

Resources

- Open Data Structures (pseudocode edition), by Pat Morin. Available online at http://opendatastructures.org
- Data Structures and Algorithms in Python, by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser. 2013. (1st. ed.). Wiley Publishing

Thanks